DEOS LoRa Telemetry Transport Adapter ICD

Sürüm 1.1 – Uzak Telemetri ve Yer İstasyonu

Emir Arkalı (emirarkali07@gmail.com)
DEOS – Dokuz Eylül Otonom Sistemler

İlk Yayın: 28 Eylül 2026 | Son Revizyon: 13 Ekim 2026

Doküman Tipi:	Interface Control Document (ICD)	**Doküman Kodu:**	`DOC-ICD-TRP-LORA-007`
Doküman Sürümü:	Sürüm 1.1	**Protokol Sürümü:**	v1.1 (`0x11`) – Medya Bağımsız
Yazar:	Emir Arkalı (`emirarkali07@gmail.com`)	**Son Değiştiren:**	Emir Arkalı
İlk Yayın Tarihi:	28 Eylül 2026	**Son Revizyon Tarihi:**	13 Ekim 2026
Durum:	Yürürlükte / Onaylandı	**Taşıma / Kapsam:**	LoRa UART Taşıma Adaptörü & CRC-CCITT

Doküman Revizyon Geçmişi

Revizyon	Tarih	Yazar / Değiştiren	Açıklama
v0.3	28 Eylül 2026	Emir Arkalı	İlk taslak ve monolitik ICD v0.3 tanımı.
v1.0	04 Ekim 2026	Emir Arkalı	LoRa telemetri adaptörü bağımsız doküman oluşturuldu.
v1.1	13 Ekim 2026	Emir Arkalı	Sürüm 1.1 standardizasyonu, kurumsal üstveri ve revizyon geçmişi eklendi.

1 Giriş ve Kapsam

Bu doküman (ICD), DEOS araç içi ağı (CAN-FD / Ethernet) ile **Yer Kontrol İstasyonu** arasında kurulan **LoRa (Uzun Mesafe Radyo)** tabanlı telemetri haberleşmesinin kurallarını, çerçeve (framing) yapısını ve hata telafi (ACK/Retry) algoritmalarını tanımlar.

Yer Kontrol yazılımını geliştirecek mühendislerin, LoRa modülüne seri port (UART) üzerinden basacakları veriyi bu dokümanda belirtilen kurallara göre şekillendirmesi zorunludur.

2 Temel Mimari ve Şeffaflık (Transparency)

DEOS mimarisinde mesajlar bellek üzerinde `deos_message_t` adlı 84 baytlık azami tampon yapısına (*8 bayt header + azami 76 bayt payload*) sahiptir. LoRa arayüzü, bu standart DEOS mesajını **hiç bozmadan** (şeffaf bir şekilde) havadan taşımakla yükümlüdür. Hava süresini (airtime) optimize etmek amacıyla bellekteki 84 baytın tamamı basılmaz; yalnızca 8 baytlık sabit başlık ile o anki `payload_len` kadar gerçek veri baytı ($8 + N$ bayt) çerçevelenerek iletilir. Yer Kontrol İstasyonu, kendi içinde bir `deos_message_t` paketi oluşturur, bunu bir "LoRa Çerçevesi (Frame)" içine hapseder ve gönderir. Araçtaki Main Node (Gateway), bu çerçeveyi çözer, içinden orijinal DEOS mesajını çıkarır ve doğrudan aracın iç ağına (Router) yönlendirir.

3 LoRa Çerçeve (Frame) Yapısı

Seri haberleşmede byte kaymalarını (misalignment) engellemek ve havadaki veri kayıplarını saptamak için her mesaj bir **Senkronizasyon Başlığı (Header)** ve **CRC16** doğrulaması ile paketlenir.

Genel paket yapısı şu şekildedir:

MAGIC_1	MAGIC_2	Flag	Length	Payload (DEOS Mesajı)	CRC16
1 Byte	1 Byte	1 Byte	1 Byte	N Byte	2 Byte
`0xAA`	`0x55`	(Bkz. Bayraklar)	N	Full DEOS Frame	CCITT

3.1 Alanların Açıklamaları

- **MAGIC_1 & MAGIC_2:** Senkronizasyon baytlarıdır. Sabit olarak `0xAA` ve `0x55` değerlerini alırlar. Alıcı taraf, UART üzerinden gelen veri akışında bu iki baytı yan yana görene kadar tüm verileri çöpe atmalıdır.
- **Flag (Bayrak):** Mesajın türünü ve ACK talebini belirtir.
- **Length:** Payload alanının uzunluğunu (bayt cinsinden) belirtir. Dinamik olarak hesaplanır (`offsetof(deos_message_t, payload) + payload_len`). ACK paketlerinde ise sadece CRC doğrulama verisi taşındığı için **2** olur.
- **Payload (Full DEOS Frame):** Esas verinin taşındığı yerdir. Bu alan `deos_message_t` yapısını taşır. Gönderim formatı şu şekildedir: Struct'ın başından `payload` dizisine kadar olan sabit kısım (Header alanı) kopyalanır, ardından sadece `payload_len` kadar dolu olan gerçek veri baytları eklenir. Kalan boş baytlar hava süresi (airtime) tasarrufu için gönderilmez.
- **CRC16:** Sırasıyla Flag, Length ve Payload alanları üzerinden hesaplanan 16-bit CRC-CCITT (Polinom: 0x1021) değeridir. *MAGIC baytları CRC hesaplamasına dahil edilmez.* (Little-endian olarak, önce LSB sonra MSB yollanır).

3.2 Flag (Bayrak) Değerleri

Makro Adı	Değer	Açıklama
`LORA_FLAG_NONE`	`0x00`	Normal bir veri paketidir. Alıcı bu paketi onaylamak (ACK göndermek) zorunda değildir.
`LORA_FLAG_ACK_REQ`	`0x01`	Alıcıdan bu paket için kesinlikle bir Alındı Onayı (ACK) beklenmektedir.
`LORA_FLAG_IS_ACK`	`0x02`	Bu paket bir onay (ACK) paketidir. Payload kısmında, onaylanan paketin CRC16 değeri (2 byte) yer alır.

4 ACK ve Tekrar Gönderme (Retry) Mekanizması

Araçtan yere veya yerden araca giden bazı hayati mesajlar (örneğin Acil Durum / E-Stop, Yörünge Gönderme vb.) havada kaybolmamalıdır. Bu tür kritik iletimlerde Yer Kontrol yazılımı şu senaryoyu işletmelidir:

4.1 Gönderici (Yer Kontrol) Davranışı

1. Yer kontrol yazılımı `deos_message_t` paketini hazırlar.
2. Paketi LoRa çerçevesine sokarken Flag alanını `0x01` (`LORA_FLAG_ACK_REQ`) olarak ayarlar ve CRC'sini hesaplayarak UART'tan basar.
3. Gönderim tamamlandığı an bir zamanlayıcı (timer) başlatılır. Maksimum bekleme süresi (Timeout) genellikle 500ms ile 1000ms arası seçilmelidir.
4. Bu süre içinde alıcıdan `LORA_FLAG_IS_ACK` bayrağına sahip bir onay paketi gelmezse, veya gelen onay paketinin içindeki (Payload) CRC değeri, bizim gönderdiğimiz paketin CRC'si ile eşleşmezse; paket **kayıp** sayılır.
5. Kayıp durumunda aynı çerçeve tekrar yollanır. Bu işlem belirlenen maksimum tekrar sayısına (Örn: 3 kez) kadar denenir.

4.2 Alıcı (Araç - Main Node) Davranışı

1. Araçtaki sistem, UART üzerinden gelen baytları dinler. `0xAA 55` sekansını yakaladığında okumaya başlar.
2. Gelen Flag değerini okur, ardından Length değerine göre Payload'ı okur, sonunda CRC16 değerini kendi hesapladığı CRC ile karşılaştırır. Eşleşmezse paketi yok sayar.

3. Eğer eşleşirse ve Flag değeri `0x01` (`ACK_REQ`) ise; araç hiç vakit kaybetmeden (DEOS Router'a mesajı iletmeden önce) bir ACK paketi hazırlar.

4. ACK Paketinin yapısı:
   `[0xAA] [0x55] [Flag: 0x02] [Length: 0x02] [Gelen Mesajın CRC'si (2 Byte)] [Yeni CRC16 (2 Byte)]`
   şeklindedir ve derhal Yer Kontrol'e geri yollanır.

5 Örnek CRC-CCITT (0x1021) C Algoritması

Yer Kontrol yazılımını geliştirecek taraf (Python, C++, C# vb.), paket bütünlüğünü kontrol etmek için standart CCITT algoritmasını kullanmalıdır. C tabanlı bir örnek aşağıda verilmiştir:

```
uint16_t crc16_ccitt(uint16_t crc, const uint8_t *src, size_t len)
{
    while (len--) {
        crc = crc ^ (*src++ << 8);
        for (int i = 0; i < 8; i++) {
            if (crc & 0x8000)
                crc = (crc << 1) ^ 0x1021;
            else
                crc = crc << 1;
        }
    }
    return crc;
}
```

Not: Yukarıdaki fonksiyona başlangıç değeri (seed) olarak `0xFFFF` verilmelidir. Hesaplamaya `Flag` baytından başlanmalı, ardından `Length` ve Payload baytları ile devam edilmelidir. MAGIC baytları atlanmalıdır.